Cloud Computing Mini Project - Report

Student: Eesh Saxena **Roll:** 230101032 **Repo:** https://github.com/eeshsaxena/cs3081-cloud-mini-project **Final tag:** `final`

0. Architecture overview

The system has four planes on one k3d (k3s-in-Docker) node: **storage** (SeaweedFS, S3 API, namespace `data`), **pipeline** (labelling workers, namespace `pipeline`), **serving** (FastAPI `/parse`, namespace `app`, behind a Traefik Ingress), and **control** (Makefile + GitHub Actions). The teacher (`phi3.5` on Ollama) labels commands; `clean.py` produces `datasets/v1/train.jsonl`; `pythia-70m` is fine-tuned on those labels; weights live in object storage and a GitHub Release (never in git or the image); the service downloads them at start-up. Data flow: `commands_pool -> teacher shards -> clean -> train.jsonl -> student -> models/ -> service -> /parse`.

1. Task 1 - Object storage & parallel teacher labelling

Architecture. `datasets` and `models` buckets on SeaweedFS. An **indexed Job** runs two workers; Kubernetes gives each a `JOB_COMPLETION_INDEX` used as `SHARD_INDEX` (`SHARD_COUNT=2`). **Implementation.** `worker/label_worker.py` takes commands where `line_number % SHARD_COUNT == SHARD_INDEX`, calls Ollama (temp 0, seed 42) with `prompts/teacher_prompt.txt`, and writes `{"id","text","teacher_raw"}` to `datasets/teacher/shard-N.jsonl` (raw reply stored unedited, with resume + periodic upload). `worker/clean.py` removes (a) invalid JSON, (b) wrong keys, (c) bad intent, (d) slot value not present in the command, (e) duplicate text (smallest id kept), writing `datasets/v1/train.jsonl` in id order with its SHA-256. **Results.**

input	(a) invalid	(b) keys	(c) intent	(d) slot	duplicates	kept
3000	0	1	3	86	49	**2861**

Checksum `datasets/v1/train.sha256` = **`5dfb8eb816a50909c5aff4fe30e9e6557c2f71160c91641a9f93478a137a24a3`**. (`input = a+b+c+d+duplicates+kept` -> `0+1+3+86+49+2861 = 3000`, verified; checksum is deterministic on re-run.) With JSON-constrained decoding (`format: "json"`) the teacher produced **100 %** valid JSON and **0** empty replies over all 3,000 commands, so rejections (a) are zero and cleaning is dominated by (d) invented-slot and duplicate removal. **Sharding (<= ½ page).** Deterministic modulo split -> every id appears in exactly one shard, no coordination needed, and shards can run on separate pods in parallel; adding workers only changes `SHARD_COUNT`. **Limitations / improvements.** CPU teacher is slow (~**1.1 h** for 3,000 commands on 2 parallel shards, laptop CPU); a shared work queue would rebalance stragglers better than static modulo sharding.

2. Task 2 - Training the student (sequence-level distillation)

Architecture. Each example is `Command: <cmd>\nJSON: <teacher JSON>`; the student learns to reproduce the teacher's output (sequence-level). Prompt tokens are masked in the loss so only the JSON is learned. **Implementation.** `training/train.py` reads only `training/config.yaml` (seed 42, lr 5e-5, batch 8, 3 epochs, max_seq_len 128, first 1,500 records). `make train SUBSET=50` smoke-tests. Loss per epoch -> `training/loss.csv`. Model + tokenizer -> `models/v1/`, archived to `v1.tar.gz` with SHA-256; v2 uses a different seed. **Results.**

model	json_validity	intent_acc	date	time	place	full_frame
teacher (phi3.5)	100.0	85.0	80.0	88.0	85.0	**48.0**
untrained pythia-70m	0.0	0.0	0.0	0.0	0.0	**0.0**
student v1	99.0	82.0	77.0	85.0	82.0	**43.0**
student v2 (seed 7)	97.0	78.0	71.0	88.0	85.0	**44.0**

All percentages over the 100-example `validation.jsonl`. Loss per epoch (student v1): **0.257 -> 0.105 -> 0.072** (`results/loss_v1.csv`; v2: 0.245 -> 0.092 -> 0.070 in `results/loss_v2.csv`). Saved as `results/teacher_validation.json`, `results/score_base.json`, `results/score_student_v1.json`, `results/score_student_v2.json`. The untrained base emits free-form text (0 % valid JSON); after distillation the student reaches **99 % valid JSON** and **full_frame 43 %** vs the teacher's 48 % - it keeps most of the teacher's skill but a 70 M model underfits the slot rules, as expected.

Note (bug found & fixed in testing). The first training run scored 0 % because the prompt ended in a trailing space (`"JSON: "`): that space is its own token when the prompt is tokenised alone but merges into `" {"` inside the full training string, so the student emitted end-of-text immediately at inference. Ending the prompt at `"JSON:"` (completion owns the leading space) makes the prompt a clean token prefix of the target; validity went 0 % -> 99 %. Fixed identically in `train.py`, `predict.py`, and the service's `model_runner.py` so training and serving tokenise the boundary the same way. **Explanation (required).** (a) Sequence-level distillation trains on the teacher's *generated text* (its output tokens). (b) Logit-based distillation trains on the teacher's *full probability distribution* per token (soft targets). (c) It is not possible here because the teacher is served through Ollama, which returns text only - we never see its logits, and the vocabularies differ. (d) Limitation: a 70M model on ~1,500 noisy labels underfits the slot rules; errors are real, not fabricated. **Limitations / improvements.** Student full_frame (43 %) trails the teacher (48 %) - a 70M model underfits; more epochs, a larger student, or a cleaner teacher prompt to cut label noise would help. True logit-level distillation would need a same-vocabulary teacher we can read logits from.

3. Task 3 - Inference service in a container

Architecture. FastAPI, port 8000: `POST /parse`, `GET /healthz`, `GET /readyz`. Config from `MODEL_URI`, `MODEL_VERSION`, `S3_ENDPOINT`, `MAX_NEW_TOKENS`; AWS creds from a Secret. Model downloaded from object storage at start-up (loaded in a background thread -> `/readyz` 503 until ready). **Implementation.** Every output is validated against `schema.json`; invalid -> HTTP 500 `{"error":"invalid_model_output"}`, never broken JSON with 200. `FAULT_INJECT=1` forces an invalid output to test that path. Headers `X-Model-Version`, `X-Served-By` (Pod name). CPU-only pinned image, weights excluded. **Results.** Real-model inference verified **locally** (service `Runner` + `models/v1`, no Docker): **10/10** validation commands returned schema-valid 200 objects, an `"not json"` output -> `validate()` = `None` -> the 500 path, and single-request latency ~= **2.2 s** on laptop CPU (`results/service_local_inference.txt`); the containerised cluster p95 is measured under load in §7. The service also ran live under `uvicorn` with the model loaded from disk: `GET /healthz` and `GET /readyz` -> **200**, and `POST /parse` -> **200** with headers `X-Model-Version: v1` / `X-Served-By` (e.g. *weather in london* -> `{"intent":"weather_query","date":null,"time":null,"place":"london"}`; `results/service_endpoints.txt`). The image built on a Linux runner is **1.74 GB** (< 2 GB; `docker images`, `results/cluster_image_size.txt`), and in the cluster it answered `/parse` through the Ingress with the right headers (`results/cluster_ingress_parse.txt`). Under the §7 load each replica peaked at ~**460 MiB** at its 1 GiB limit (`results/cluster_top_pods.txt`) - no OOM. **Limitations / improvements.** Single uvicorn worker per replica; batching requests would raise throughput.

4. Task 4 - Kubernetes deployment, scaling, reliability

Architecture. Deployment (2 replicas, requests 250m/512Mi, limits 1/1Gi), Service, Traefik Ingress on `/parse`, HPA 1-2 @ 70% CPU. `RollingUpdate maxUnavailable=0, maxSurge=1`. All runs below were produced on a real k3d cluster (two replicas, Traefik Ingress) via the `live-cluster` workflow - the same k3s-in-Docker stack as the laptop, run on a Linux runner. **Results.** - Rolling update v1->v2 under load: **2 of 39** requests non-200 during the window (5.1%); over the larger steady-state load test the only failures were the model's ~1 % invalid-output 500s (§7), so these are most likely the same, not rollout downtime - `maxUnavailable=0` kept both replicas serving. `X-Model-Version` = **v2** during the rollout and **v1** after `rollout undo` (`results/cluster_rolling_update.txt`). - Recovery: deleted a Pod under the live Deployment; replicas returned to **2** within ~10 s (`results/cluster_recovery.txt`). - Cold start (3 runs): **8 s, 8 s, 8 s** -> median **8 s**; logs show startup -> `model loaded` -> `/readyz` 200 (`results/cluster_coldstart_logs.txt`). - Load balancing (2 Ready): **109 / 104** requests -> **51 % / 49 %** across replicas (`results/cluster_loadtest_headers.json`). - HPA: under load CPU reached **196 %** of request (target 70 %); the HPA computed a scale-up but was capped at `maxReplicas=2` (`ScalingLimited: TooManyReplicas`) (`results/cluster_hpa.txt`). **Limitations.** Single node -> no cross-machine fault tolerance; `maxReplicas=2` and both replicas share the same CPUs, so scale-out adds little real capacity; metrics-server warms up slowly (the first HPA reads were `<unknown>` before it settled).

5. Task 5 - Subnetting & NACL

Subnets of `10.20.0.0/16` (largest first, contiguous):

Subnet	Hosts	CIDR	Range	Broadcast	Usable
app	1000	10.20.0.0/22	10.20.0.1-10.20.3.254	10.20.3.255	1022
public	250	10.20.4.0/24	10.20.4.1-10.20.4.254	10.20.4.255	254
data	100	10.20.5.0/25	10.20.5.1-10.20.5.126	10.20.5.127	126
management	20	10.20.5.128/27	10.20.5.129-10.20.5.158	10.20.5.159	30

NACL (public subnet). Verified **6/6** on the sample packets (`nacl_sim.py`). Inbound: deny all from `198.51.100.0/24` (rule 100, before the web allows), allow tcp 80/443 from anywhere, tcp 22 from management, tcp 1024-65535 from app (replies). Outbound: tcp 8000 to app, tcp 1024-65535 anywhere (replies). **Explanation.** A NACL is stateless, so replies need their own rules; and because the first matching (lowest-numbered) rule wins, the `198.51.100.0/24` deny must sit below the 80/443 allows or packet S04 would be allowed. Full working in `network/subnets.md`. **Limitations / improvements.** Largest-first contiguous allocation leaves gaps between blocks and a subnet can't grow in place; defining the plan as code (Terraform/CDK) would make it reproducible. A NACL is coarse (subnet-wide and stateless); per-instance, stateful security groups would be finer.

6. Task 6 - Reproducibility, testing, CI/CD

`make up` builds + deploys from a fresh clone in <= 3 commands; `make down` deletes the cluster. CI stages: lint -> tests (6 test files, 22 tests) -> model evaluation from the `model-v1` Release -> quality gate `score.py --min-validity 95` -> gitleaks (reports nothing). **On the trained model the pipeline is green** (the evaluate job scored the Release model at 99 % validity >= 95). The deliberately-failing run on the untrained base (gate at 0 % validity) is run 41300788338: https://github.com/eeshsaxena/cs3081-cloud-mini-project/actions/runs/41300788338. `AI_USE.md` records AI usage. **Limitations / improvements.** The cluster integration test runs on a single-node k3d (via the `live-cluster` workflow), not a true multi-node cluster, and test coverage isn't gated. Adding a coverage threshold and a smoke deploy on every PR would harden the pipeline further.

7. Task 7 - Performance, cost, review

Load test (cluster, standard command). Locust, 4 users, 90 s, against the Traefik Ingress (2 replicas, 1 vCPU / 1 GiB each): **213 requests**, **2 failures (0.93 %)** - both HTTP 500 from the model's ~1 % invalid-output path (returned by design, never broken JSON with 200) - p50 **1600 ms**, p95 **3300 ms**, p99 3600 ms, throughput **2.41 rps** (`results/cluster_loadtest_stats.csv`). Each replica used ~**0.95 vCPU** and ~**460 MiB** under load (`results/cluster_top_pods.txt`); cold start median **8 s**. (An indicative single-process local run is in `results/loadtest_local_*`.) **Cost.** *Assumption:* 2 replicas, each billed at limits 1 vCPU + 1 GiB. hourly = 2 x (1x3.50 + 1x0.45) = **7.90 INR/hr**; at the measured **2.41 rps** -> 8 676 req/hr -> cost/1000 = **0.91 INR** (`results/cost_cluster.txt`, `scripts/cost.py`). **SLO.** *Target:* >= 99 % of requests succeed over the load-test window. Cluster run: **211/213 (99.1 %)** succeeded; the 2 failures are the model's invalid-output 500s, within the 1 % error budget (2.13 of 213). The service never returned broken JSON with 200. **Review table.**

Component	Evidence (measured)	Limitation	Improvement
Data & storage	2861 kept of 3000	teacher noise	better prompt / larger teacher
Model quality	full_frame 43% (student v1)	70M underfits	more epochs / bigger student
Latency & throughput	p95 3300 ms, 2.41 rps (cluster)	CPU inference	request batching
Resource utilisation	~0.95 vCPU, ~460 MiB / pod	shared node	real multi-node cluster
Reliability & recovery	recovery ~10 s to 2 replicas, cold 8 s	single node	multi-node, PodDisruptionBudget
Security	nacl 6/6, gitleaks clean	permissive S3 identity	per-service scoped keys

References

SLURP (Bastianelli et al., EMNLP 2020); Kim & Rush, Sequence-Level KD (EMNLP 2016); k3d, SeaweedFS, Hugging Face Transformers, Kubernetes, AWS VPC NACL docs.